

Network Attack: Architectural Decision Record

Sania Ali, Madeline Geittmann, Jackson Horan, Lara Seledotis

Dr. Manar Mohaisen

COMP 379: Machine Learning

28 April 2026

1. Executive Summary:

1.1 Overview

An intrusion detection system (IDS) is a security tool that monitors network traffic and device activity, looking for suspicious or potentially malicious activity as well as security violations [2]. IDSs can detect security threats, but cannot stop them on their own. For this reason, they are often incorporated with intrusion prevention systems (IPS), which can automatically act to prevent attacks. IDSs use signature- and anomaly-based detection as their primary methods for threat detection [15]. Signature-based detection analyzes network packets to find characteristics or behaviors that signify an attack. Contrarily, anomaly-based detection methods utilize machine learning to continuously update and refine a model that represents baseline, normal network activity. It subsequently compares network activity to this standard, flagging any traffic that deviates. Anomaly-based detection is prone to false positives, though, so it performs best alongside other detection methods.

Regardless of which detection methods are implemented in an IDS, when a threat or malicious activity is detected, the IDS alerts an incident response team and records relevant information about the attack. Logs help to refine and update the IDS's detection criteria to optimize future performance. IDSs are useful in detecting many types of threats, but hackers can often get around their solution tactics. Therefore, it is crucial to pair an IDS with another security solution to maximize their performance. However, in this project we will be focusing specifically on creating an effective, robust, and generalizable IDS that detects malicious network activity.

1.2 Data

We used a dataset (CICIDS2017) from the University of New Brunswick, which contains common, up-to-date, labeled network attacks and benign traffic [5]. This dataset was artificially constructed to model real-world network traffic, prioritizing capturing abstract human behavior and interactions and naturalistic benign traffic. The authors identified several criteria that they deemed necessary in building a reliable dataset for IDS. Data was captured over the course of five days; the first contains only benign traffic, and the latter four days contain both benign and attack network activity.

The dataset was modified to contain only Wednesday traffic, including four different types of denial-of-service (DoS) attacks, benign traffic, and Heartbleed attacks. A DoS attack occurs when legitimate users are unable to access a network's resources due to the actions of a malicious actor [1]. DoS conditions act by flooding a network with traffic, so much so that it cannot respond to legitimate users, and sometimes causing a crash. These types of attacks are detrimental because during a crash, users are unable to access a network's resources, leading to potential loss of revenue and trusted customers. Also, crashes require time and money to be spent to identify and resolve the attack and make changes to prevent future attacks from occurring. Therefore, having an IDS placed in a network to monitor traffic flow can be crucial to maintain network integrity and learn from previous attacks to prevent crashes from occurring.

1.3 Goals

The major goal of our project was to build a classifier that distinguishes benign network traffic from DoS attacks. While we do not want benign traffic to be incorrectly labeled as malicious, it is more crucial to prevent any network attacks from leading to a false negative classification. To start, all Heartbleed attacks were removed from the dataset, as per project instructions because these are not considered DoS-type attacks. DoS attacks were grouped together, allowing creation of a binary classifier: benign or DoS attack (0 or 1). We aimed to identify the features within this dataset that have the strongest predictive ability in identifying traffic type. We focused on creating a balance between model performance and complexity to create the best model to be deployed for usage.

This classifier must be robust and have strong predictive accuracy while maintaining generalizability. An ideal model can be generalized to DoS attacks that are not included in the CICIDS2027 dataset; new methods of network attacks are continuously being refined by hackers, so the ability to generalize our model after time is crucial. Overall, we aim to produce a classifier that incorporates multiple network feature types without reducing generalizability. In the future, it will be crucial to continuously test the model on additional, more up-to-date data to maintain the model's performance and prevent any DoS attacks from affecting a network.

2. Data Sources:

2.1 Context & Technology Choice

This project uses a modified version of the CIC-IDS2017 dataset which looks only at network traffic on Wednesdays. The dataset was developed by the Canadian Institute for Cybersecurity, which includes both benign traffic and several denial-of-service (DoS) attack types: DoS GoldenEye, DoS Hulk, DoS slowhttptest, and DoS slowloris. Instances labeled as Heartbleed are excluded from the dataset as they do not represent DoS attacks, and the remaining attack types are grouped into a single “attack” class to form a binary classification problem alongside the “benign” class. The dataset represents network traffic as flows, where each flow corresponds to a communication session defined by attributes such as source and destination IP addresses, ports, protocol (TCP/UDP), and timestamps. The dataset is described by 78 extracted features capturing statistical and behavioral characteristics of the traffic. The data is provided in multiple formats, including: CSV, JSON, and Parquet files. This allows flexibility in data access and processing, with the final column in each file representing the class label indicating whether a given flow is benign or an attack.

2.2 Justifications

The CIC-IDS2017 dataset was chosen for its realistic network traffic and labeled DoS attacks. Wednesday data was used to isolate relevant attack scenarios while reducing noise.

Non-DoS instances (ex: Heartbleed) were removed, and attack types were combined into a single class to enable binary classification and simplify the task. The flow-based structure and rich feature set support effective modeling. The multiple file formats also improve flexibility in data processing.

2.3 Status

Madeline Geittmann: Proposed, 4/10/26

Madeline Geittmann, Sania Ali, Lara Seledotis: Accepted, 4/17/26

Completed & passed onto team, 4/24/26

Data Engineering Pipeline

3. Data Integration (Extract)

3.1 Context & Technology Choice

The extraction process for our project used `read_csv/read_json/read_parquet` commands to extract the data from all individual csv/json/parquet files inside our folder. Our team created a separate “data” folder to store all of our pre-processed csv files for easy extraction. For the json files we used “`lines = True`” parameter to enable the extraction to be in line. This ensures that the file is extracted one line at a time, rather than as a whole json object at once, this is best when handling large data sources such as our files. Additionally, for the parquet files the parameter `engine=pyarrow` is used inside `read_parquet()` as this arrow table format is best in speed, performance, and memory when handling large amounts of data.

```
dataframes = {
    "ids_0": ids_0, "ids_1": ids_1, "ids_2": ids_2, "ids_3": ids_3,
    "ids_4_": ids_4_, "ids_4": ids_4, "ids_5": ids_5, "ids_6": ids_6,
    "ids_7": ids_7, "ids_8": ids_8, "ids_9": ids_9, "ids_10": ids_10,
    "ids_11": ids_11
}
```

Figure 1: Dictionary of Extracted Files

After each file was read and extracted, we added it to a dictionary called “dataframes” that held each extracted file and the label, shown in figure 1. Then we compiled all the individual files into one large dataframe. This was done by using `pd.concat` and parameters “`join=inner`”

and “ignore_index=True” so the combined dataframe would only stack the columns which exist in every file. This is important to avoid gaps in the code and ensure that the data stays in proper positioning before transformations.

For the extraction section the main technology choices were the pandas library and the various functions listed below:

- read_csv()
- read_json()
- read_parquet()
- pd.concat()

3.2 Justifications

Our team decided on extracting the data through this process to safely handle the large dataset at the fastest speed possible. We used specific parameters such for the read_csv, read_json, and read_parquet to maintain safe handling of the data while optimizing speed. This is incredibly important as we are working with very large datasets. To avoid memory errors or crashing of our operating systems, proper safe handling is very important. Additionally, we decided to put all of the individual files into a dictionary so if we wanted to later identify one at a time we could easily pinpoint an individual file from the dictionary named *dataframes*. Furthermore, using pandas’ *concat* function was imperative to combine all of our individual files into a large dataframe for our later transformations, regressions, processing, and machine learning pipelines. In this process, we made sure to using the parameters applicable to only stack columns that exist in every file to minimize gaps. Each action was taken to maximize speed and safety for a large dataset.

3.3 Status:

Lara Seledotis: Proposed, 4/7/2026

Lara Seledotis: Accepted, 4/7/2026

Completed & passed onto team, 4/11/2026

4. Data Transformation (Transform)

4.1 Context & Technology Choice

The transformation section of the code takes the dataframe df_all which is the raw data compiled from the network attach data. The code performs various transforms to clean the data

and output the dataframe “clean_df.” After reviewing the data, we wanted to make the most efficient transforms possible to achieve clean and processed data before conducting the exploratory data analysis (EDA) machine learning pipeline. The transforms include stripping null or void columns, replacing spaces with underscores, replacing backslashes with *per*, converting all strings to numeric values, and replacing infinity with NaN [3]. We also dropped the flow_packets_per_s rows where that value was less than zero as we cannot have a negative flow rate. The code also removes the Heartbleed column and changes labels to binary values. These transformations are important so our team can work with clean, filtered data and gather meaningful connections from it. Using the transformed data we can collect meaningful statistics such as regression data, distribution and averages without unwanted biases impacting our analysis.

For the transformation section the main technology choices were the pandas library and the various functions listed below:

- df_all.shape()
- col.strip()
- col.replace()
- pd.to_numeric()
- df_all.replace()
- df_all.info()
- df_all.dropna()

4.2 Justification

The technology choices used are imperative for cleaning the data and filtering the unwanted values in the rows and columns of our large dataframe. We made sure to copy the dataframe given from the extract section, *df_all*, into a new dataframe named *clean_df*. This was to ensure that dropping any values and all the transforms would not overwrite our original dataframe in case we wanted to compare the two later. We used the functions above to clean the column names, replace hard to interpret characters such as spaces and slashes. Additionally, we removed extraordinarily large or unwanted numbers such as infinite values, negative flow rates, and NaN values so they would not skew our data statistics in the Exploratory Data Analysis section.

4.3 Status

Lara Seledotis: Proposed, 4/7/2026

Lara Seledotis: Accepted, 4/7/2026

Completed & passed onto team, 4/11/2026

5. Data Storage (Load)

5.1 Context & Technology Choice

The load section of the network attack project loads the clean data named *clean_df* into a csv file, json file and parquet file for the next step of our project, the EDA. The data was loaded into these three file types so our team could effectively choose which would be the best file type for the later steps of the process such as EDA, data pre-processing, feature engineering, model selection, model testing and model training. The final clean loaded files are named “cleaned_raw_data.csv,” “cleaned_raw_data.json,” and “cleaned_raw_data.parquet.” The first 20 lines of the CSV and JSON file is shown below, where we can see the clean raw data after transforms and before processing.

```

1 Destination_Port,Flow_Duration,Total_Fwd_Packets,Total_Backward_P
2 53,61205,4,2,136,428,34,34,34.0,0.0,214,214,214.0,0.0,9214.93342,
3 53,222,2,2,90,172,45,45,45.0,0.0,86,86,86.0,0.0,1180180.18,180180.
4 53,23759,2,2,70,126,35,35,35.0,0.0,63,63,63.0,0.0,8249.505451,168
5 80,401,2,0,12,0,6,6,6.0,0.0,0,0,0.0,0.0,29925.18703,4987.531172,4
6 57406,4,2,0,12,0,6,6,6.0,0.0,0,0,0.0,0.0,3000000.0,500000.0,4.0,0
7 57026,4,1,1,6,6,6,6,6.0,0.0,6,6,6.0,0.0,3000000.0,500000.0,4.0,0
8 53,48317,2,2,76,120,38,38,38.0,0.0,60,60,60.0,0.0,4056.543246,82
9 53,211,2,2,72,334,36,36,36.0,0.0,167,167,167.0,0.0,1924170.616,18
10 53,226,2,2,64,168,32,32,32.0,0.0,84,84,84.0,0.0,1026548.673,17699
11 443,5168505,8,6,627,3268,342,0,78.375,127.6378107,1460,0,544.6666
12 443,61141645,15,15,688,4292,363,0,45.86666667,105.454707,1448,0,2
13 443,1,1,1,6,6,6,6,6.0,0.0,6,6,6.0,0.0,12000000.0,2000000.0,1.0,0
14 21,134826,20,25,176,440,43,0,8.8,12.68069399,52,0,17.6,16.1606518
15 53,200,2,2,76,282,38,38,38.0,0.0,141,141,141.0,0.0,1790000.0,2000
16 443,2521187,14,9,1511,7279,517,0,107.9285714,191.4714135,2166,0,8
17 53,47796,1,1,92,174,92,92,92.0,0.0,174,174,174.0,0.0,5565.319274,
18 443,30788265,12,13,1251,4795,861,0,104.25,247.9582535,1460,0,368
19 443,60420671,14,13,1352,4818,492,0,96.57142857,176.5279295,1858,0
20 53,23420,1,1,40,56,40,40,40.0,0.0,56,56,56.0,0.0,4099.060632,85.3

```

Figure 2: Clean Data CSV File

```

1 {"Destination_Port":53,"Flow_Duration":61205,"Total_Fwd_Packets":
2 {"Destination_Port":53,"Flow_Duration":222,"Total_Fwd_Packets":2,
3 {"Destination_Port":53,"Flow_Duration":23759,"Total_Fwd_Packets":
4 {"Destination_Port":80,"Flow_Duration":401,"Total_Fwd_Packets":2,
5 {"Destination_Port":57406,"Flow_Duration":4,"Total_Fwd_Packets":2
6 {"Destination_Port":57026,"Flow_Duration":4,"Total_Fwd_Packets":1
7 {"Destination_Port":53,"Flow_Duration":48317,"Total_Fwd_Packets":
8 {"Destination_Port":53,"Flow_Duration":211,"Total_Fwd_Packets":2,
9 {"Destination_Port":53,"Flow_Duration":226,"Total_Fwd_Packets":2,
10 {"Destination_Port":443,"Flow_Duration":5168505,"Total_Fwd_Packet
11 {"Destination_Port":443,"Flow_Duration":61141645,"Total_Fwd_Packe
12 {"Destination_Port":443,"Flow_Duration":1,"Total_Fwd_Packets":1,"
13 {"Destination_Port":21,"Flow_Duration":134826,"Total_Fwd_Packets"
14 {"Destination_Port":53,"Flow_Duration":200,"Total_Fwd_Packets":2,
15 {"Destination_Port":443,"Flow_Duration":2521187,"Total_Fwd_Packet
16 {"Destination_Port":53,"Flow_Duration":47796,"Total_Fwd_Packets":
17 {"Destination_Port":443,"Flow_Duration":30788265,"Total_Fwd_Packe
18 {"Destination_Port":443,"Flow_Duration":60420671,"Total_Fwd_Packe
19 {"Destination_Port":53,"Flow_Duration":23420,"Total_Fwd_Packets":
20 {"Destination_Port":80,"Flow_Duration":25382356,"Total_Fwd_Packet

```

Figure 3: Clean Data JSON File

After the ETL pipeline where the data is extracted, transformed, and loaded into csv, json and parquet files, we decided to log the clean data into the log file *etl_log_file.txt*. Below is the

image of the log file after the ETL pipeline. We decided to include it here so we can verify the pipeline execution and record it for our results.

```
comp_project1_379 > ≡ etl_log_file.txt
1 2026-04-11-14:43:40.735965, Final shape: (62995, 79)
2 2026-04-11-14:57:53.820380, Final shape: (62995, 79)
3 2026-04-11-15:09:07.138014, Final shape: (62995, 79)
4 2026-04-11-15:09:43.355543, Final shape: (62995, 79)
5 2026-04-11-17:54:23.289096, Final shape: (62995, 79)
6 2026-04-11-17:54:34.230410, Final shape: (62995, 79)
7 2026-04-11-18:13:08.298700, Final shape: (62995, 79)
8 2026-04-27-17:34:14.032228, Final shape: (62995, 79)
9
```

Figure 4: ETL Pipeline Logfile

For the loading section the main technology choices and functions used are listed below:

- .to_csv()
- .to_json()
- .to_parquet()
- Datetime library

5.2 Justification

For the loading section we wanted to load our dataframe into csv, json, and parquet files so we would have a greater variety to choose from in later processing steps. Each file type has advantages and disadvantages for certain tasks. For example JSON files are better at handling hierarchical data of various types, it is also a nested structure so the readability can be a bit clearer. However, JavaScript Object Notation (JSON) files are larger and often have slower parsing speeds, this can be incredibly impactful for our project as we are working with very large datatypes. Alternatively, Comma Separated Values (CSV) files are smaller and offer higher parsing speeds, making it the better choice for larger datasets. However, CSV values need manual parsing and cannot handle nested data. In our EDA section we will be examining the data and interpreting it, that is why we wanted multiple options. We also wanted to log our findings to record the shape and timestamp of our ETL pipeline conclusion to maintain proper documentation of the process, as was recommended in one of our in class labs [8].

5.3 Status

Lara Seledotis: Proposed, 4/7/2026

Lara Seledotis, Madeline Geittmann: Accepted, 4/7/2026

Completed & passed onto team, 4/11/2026.

6. Reading Data

6.1 Context & Technology Choice

The “clean_df” Pandas dataframe, created in the transformation section, was used for the following exploratory data analysis and machine learning pipeline. This cleaned dataframe had 62,995 samples, 78 features, and 1 label.

6.2 Justifications

“Clean_df” was the final, concatenated dataframe before exporting into various filetypes. Therefore, this dataframe was the simplest to use as it provided a seamless transition to the performance of several analyses using pandas methods and features.

6.3 Status

Lara Seledotis: Proposed, 4/7/2026

Lara Seledotis, Madeline Geittmann: Accepted, 4/11/2026

7. Exploratory Data Analysis (EDA)

7.1 Context & Technology Choice

Exploratory data analysis was performed on the cleaned dataset both before and after splitting the features and label into separate dataframes [4]. To analyze variance, the features that have more than 10,000 unique values were identified, as well as those with variance below 1. The latter should be removed during feature selection. Features appeared in categories, with multiple descriptive statistics for each feature subtype.

Univariate analysis and visualization was performed on many features to determine which features within a category are most informative. Histograms and boxplots were used to visualize features’ distributions; many had a right skew with many high outliers (Fig. 5). Bivariate analysis revealed features with very strong correlations, some of which will be dropped during feature selection as they do not provide any additional information for the model. Also, correlation matrices (Fig. 6 & 7) and pair-plots were used to visualize relationships between features within a category (Fig. 8), or between different feature categories’ descriptive statistics

(Fig. 9). Multivariate analysis and visualization included performing k-means cluster analysis on a subset of features (Fig. 10).

7.2 Justifications

Univariate and bivariate analysis helped with visualization of variance, means, and outliers as well as correlation between seemingly related features. Multivariate analysis was somewhat ineffective because of the skewness of several variables, as shown by silhouette scores between 0.3 and 0.5 depending on features chosen. Importantly, silhouette scores are not very informative for data clusters with irregular shapes [6], which is likely the case here due to the general skewness of the data. High dimensionality of the dataset also made cluster analysis less effective. However, looking at cluster analyses across variables gave insight into patterns between particular features and the label.

7.3 Status

Madeline Geittmann: Proposed 4/13/2026

Madeline Geittmann, Sania Ali, Jackson Horan: Accepted 4/19/2026

Completed & passed onto group, 4/19/2026

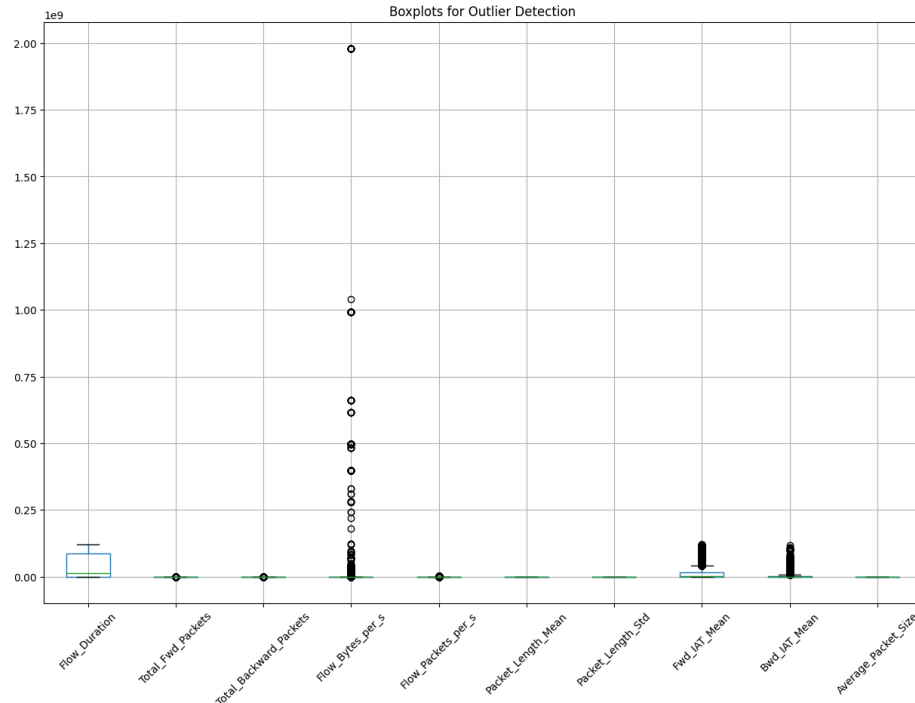


Figure 5: Boxplots of Selected Features for Outlier Detection



Figure 6: Correlation Matrix Between All Features ($n_{\text{features}}=78$)

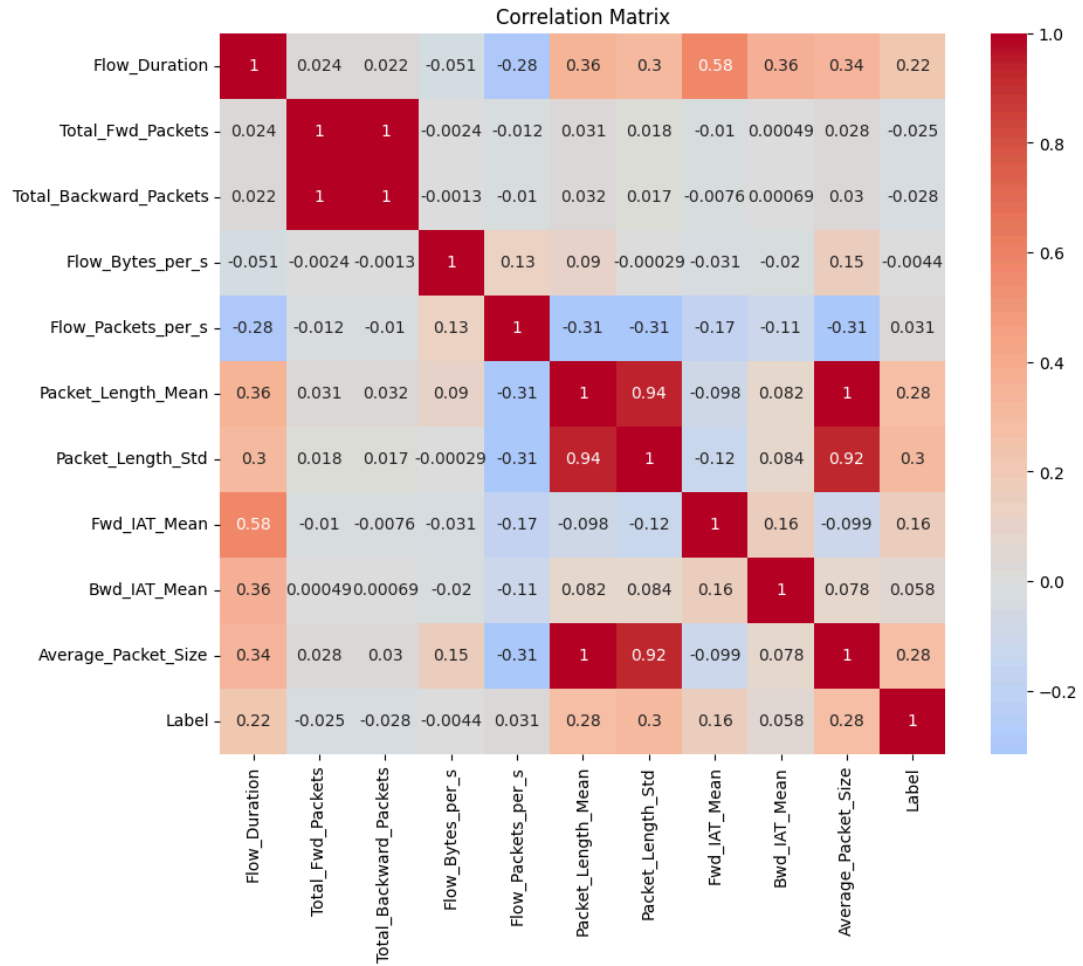


Figure 7: Correlation Matrix Between Selected Features & Label ($n_{\text{features}}=11$)

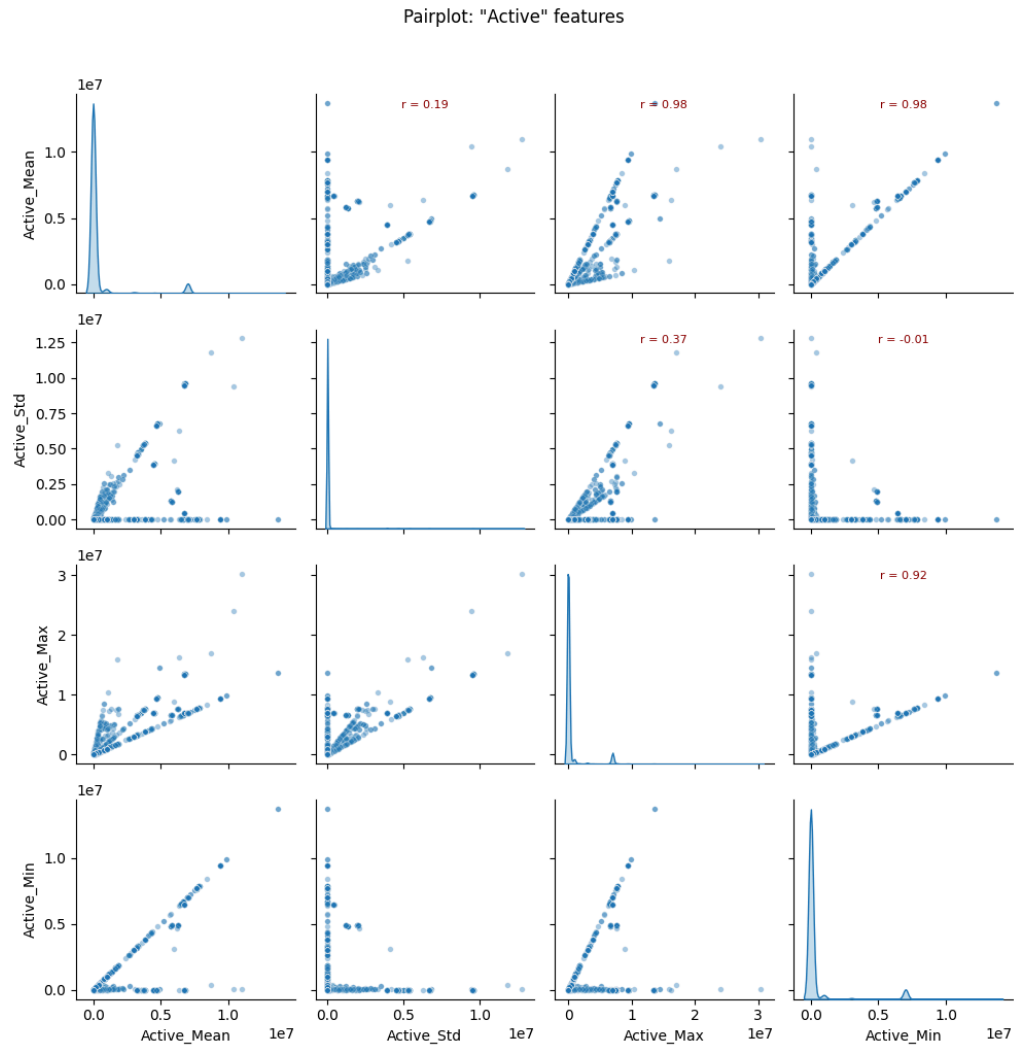


Figure 8: Correlation Pairplot Between "Active" Features

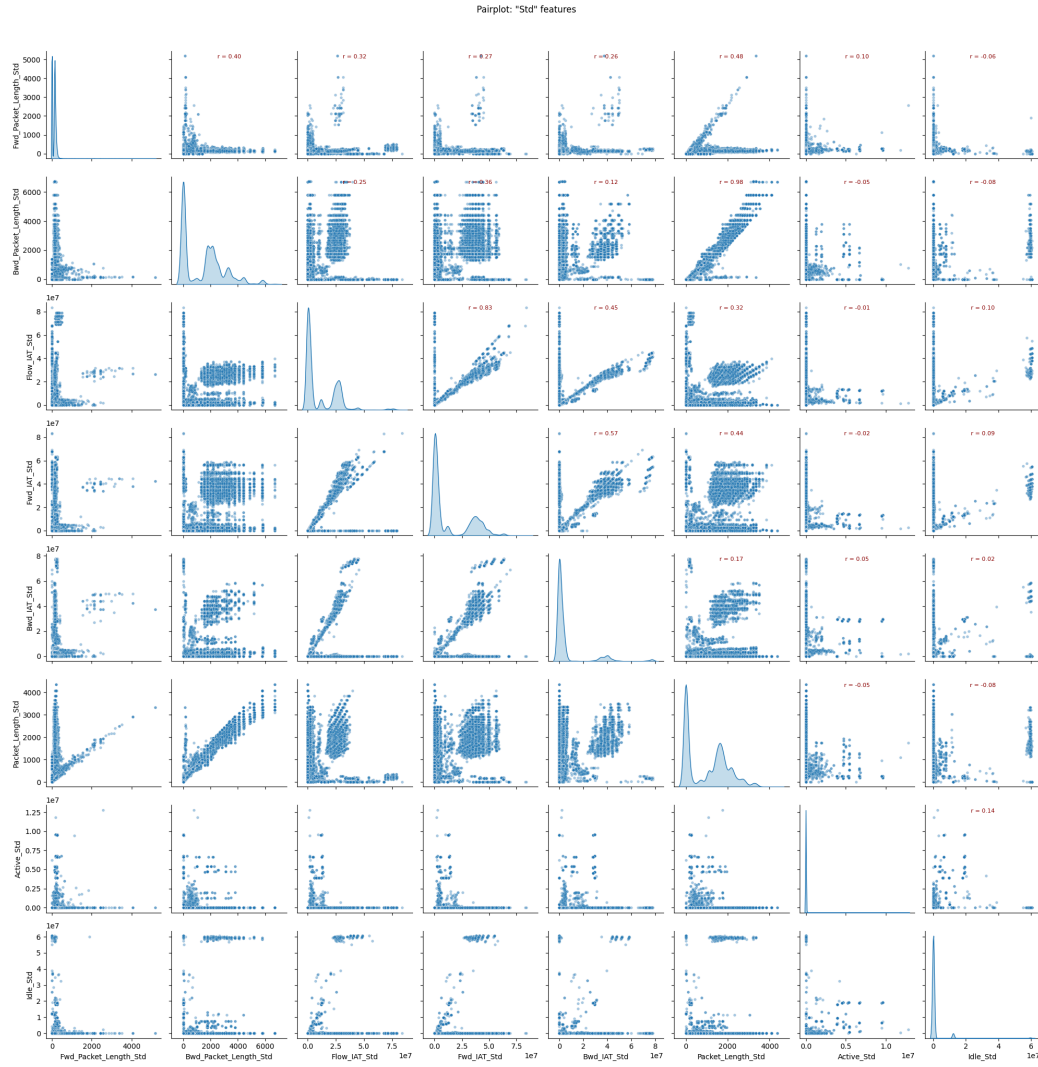


Figure 9: Correlation Pairplot Between “Std” Features

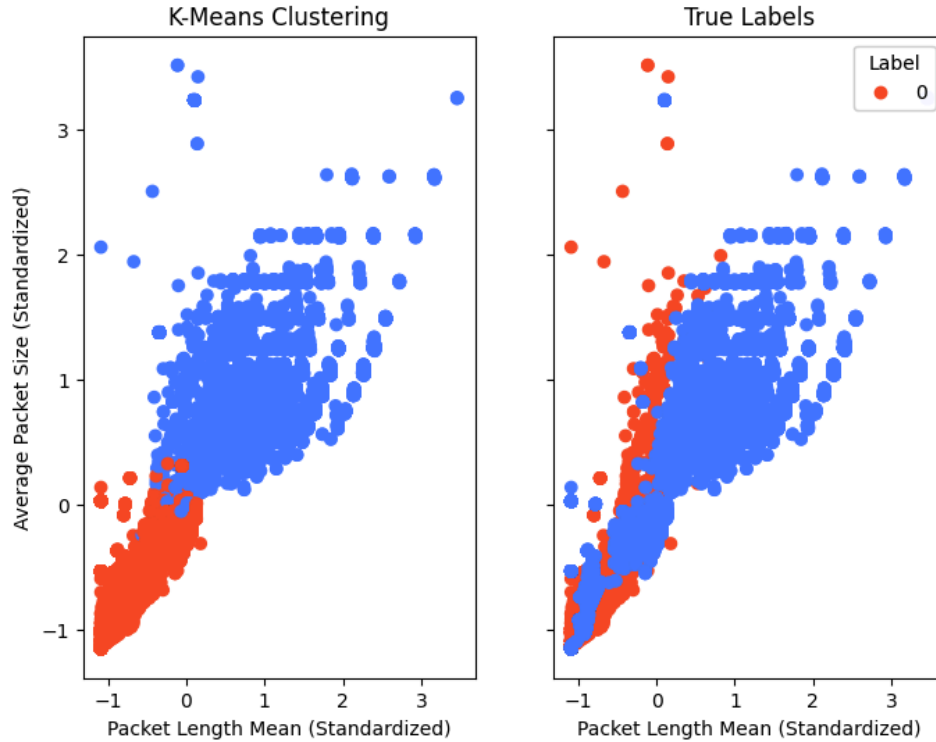


Figure 10: K-Means Cluster Analysis

Machine Learning Pipeline

8. Data Preprocessing

8.1 Context & Technology Choice

Before splitting the data, multiple steps were taken to prevent data leakage. First, all duplicate entries were dropped from the dataset, which amounted to 13,508 samples dropped. The order of the rows in the dataset were shuffled to ensure randomness. Next, the data was split into a dataframe containing all 78 features and a single array containing the target label. This dataset had around 50,000 samples after dropping duplicates, so the data was split using a 70-15-15 ratio (70.04% training set, 14.96% validation, 14.96% testing) [4].

8.2 Justifications

The validation dataset was created to assist with feature selection, as the exploratory data analysis was not definitive in which features should be kept or dropped. The split data sets are stratified due to imbalance in the label.

8.3 Status

Madeline Geittmann, Sania Ali: Proposed, 4/19/2026

Madeline Geittmann, Sania Ali: Accepted, 4/19/2026

Completed & passed onto team, 4/20/2026

9. Feature Engineering

9.1 Context & Technology Choice

In the feature engineering stage, the dataset was first examined to understand the distribution of each predictor. The mean and standard deviation of every feature in the training set was computed and compiled into a summary table using a pandas dataframe. This revealed that several features had zero variance, meaning they remained constant across all observations. Because such features do not contribute any meaningful information for distinguishing between classes, they were removed from the training, validation, and test sets. This step reduced the dimensionality of the dataset and eliminated unnecessary predictors that could potentially introduce noise into the modeling process.

```
Zero variance features:
Index(['Bwd_PSH_Flags', 'Fwd_URG_Flags', 'Bwd_URG_Flags', 'CWE_Flag_Count',
      'Fwd_Avg_Bytes_per_Bulk', 'Fwd_Avg_Packets_per_Bulk',
      'Fwd_Avg_Bulk_Rate', 'Bwd_Avg_Bytes_per_Bulk',
      'Bwd_Avg_Packets_per_Bulk', 'Bwd_Avg_Bulk_Rate'],
      dtype='object')
```

Figure 11: Zero Variance Features

Following this, the remaining features were standardized using a StandardScaler. Each variable was transformed to have a mean of approximately zero and a standard deviation of one. This ensured that all features were on a consistent scale. The scaler then was fit on the training data and then applied to the validation and test sets to maintain consistency and prevent data leakage.


```

1 scaler = StandardScaler()
2
3 X_train_scaled = scaler.fit_transform(X_train)
4 X_val_scaled = scaler.transform(X_val)           # scale the features
5 X_test_scaled = scaler.transform(X_test)
6
7
8 X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns)
9 X_val_scaled = pd.DataFrame(X_val_scaled, columns=X_train.columns)
10 X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_train.columns)

```

Figure 12: Scaling the Features

To further refine the feature set, multiple feature selection methods were applied. An L1-regularized logistic regression (Lasso) model was used to perform embedded feature selection by shrinking less important coefficients to zero. The Lasso feature selection found 41 predictors to have significance.

```

Selected features with Lasso: 41
Index(['Destination_Port', 'Total_Length_of_Fwd_Packets',
      'Fwd_Packet_Length_Max', 'Fwd_Packet_Length_Min',
      'Fwd_Packet_Length_Std', 'Bwd_Packet_Length_Max',
      'Bwd_Packet_Length_Min', 'Bwd_Packet_Length_Mean',
      'Bwd_Packet_Length_Std', 'Flow_Bytes_per_s', 'Flow_IAT_Mean',
      'Flow_IAT_Std', 'Fwd_IAT_Total', 'Fwd_IAT_Mean', 'Fwd_IAT_Min',
      'Bwd_IAT_Total', 'Bwd_IAT_Mean', 'Bwd_IAT_Min', 'Fwd_PSH_Flags',
      'Fwd_Header_Length', 'Bwd_Header_Length', 'Fwd_Packets_per_s',
      'Bwd_Packets_per_s', 'Min_Packet_Length', 'Packet_Length_Std',
      'FIN_Flag_Count', 'SYN_Flag_Count', 'PSH_Flag_Count', 'ACK_Flag_Count',
      'URG_Flag_Count', 'Down_per_Up_Ratio', 'Average_Packet_Size',
      'Avg_Bwd_Segment_Size', 'Fwd_Header_Length.1', 'Subflow_Fwd_Bytes',
      'Init_Win_bytes_forward', 'Init_Win_bytes_backward',
      'min_seg_size_forward', 'Active_Mean', 'Active_Std', 'Idle_Std'],
      dtype='object')

```

Figure 13: Selected Features with Lasso

Additionally, tree-based models such as Random Forest and XGBoost were trained to compute feature importance scores based on how much each feature contributed to reducing prediction error. Features exceeding a specified importance threshold were retained from these models. Random Forest gave 29 predictors of significance and XGBoost gave 7 predictors.

```

Selected features with Random Forest: 29
0      Destination_Port
35      Min_Packet_Length
20      Fwd_IAT_Total
21      Fwd_IAT_Mean
23      Fwd_IAT_Max
11      Bwd_Packet_Length_Min
56      Init_Win_bytes_forward
57      Init_Win_bytes_backward
54      Subflow_Bwd_Packets
16      Flow_IAT_Mean
3      Total_Backward_Packets
15      Flow_Packets_per_s
34      Bwd_Packets_per_s
7      Fwd_Packet_Length_Min
5      Total_Length_of_Bwd_Packets
32      Bwd_Header_Length
37      Packet_Length_Mean
13      Bwd_Packet_Length_Std
33      Fwd_Packets_per_s
12      Bwd_Packet_Length_Mean
31      Fwd_Header_Length
18      Flow_IAT_Max
50      Avg_Bwd_Segment_Size
51      Fwd_Header_Length.1
14      Flow_Bytes_per_s
48      Average_Packet_Size
36      Max_Packet_Length
6      Fwd_Packet_Length_Max
10      Bwd_Packet_Length_Max

```

Figure 14: Selected Features with Random Forest

```

Selected features with XGBoost: 7
35      Min_Packet_Length
0      Destination_Port
7      Fwd_Packet_Length_Min
3      Total_Backward_Packets
13      Bwd_Packet_Length_Std
9      Fwd_Packet_Length_Std
34      Bwd_Packets_per_s
Name: Feature, dtype: object

```

Figure 15: Selected Features with XGBoost

9.2 Justification

We evaluated feature selection using Lasso, XGBoost, and Random Forest. Lasso provides a sparse, interpretable set of features by eliminating those with low linear contribution.

However, its linear structure limits its ability to capture non-linear relationships and interactions between predictors.

To address this limitation, we also considered XGBoost and Random Forest, which are ensemble-based methods that model complex, non-linearity and feature interactions. Using these methods alongside Lasso provided a more comprehensive and robust assessment of feature importance. This ensured that both linear and non-linear relationships are accounted for in the selected feature set.

9.3 Status

Sania Ali: Proposed, 4/20/26

Sania Ali, Jackson Horan: Accepted, 4/20/26

Completed & passed onto team, 4/22/26

10. Processed Data Loading

10.1 Context & Technology Choice

The processed data is exported to 'processed_data.csv'.

10.2 Justification

CSV is chosen for its universality and low overhead. CSV also allows the ability to manually inspect data if needed.



Figure 16: ‘processed_data.csv’

10.3 Status

Jackson Horan: Proposed, 4/24/2026

Jackson Horan: Accepted, 4/24/2026

Completed on 4/24/2026

11. Model Selection & Training

11.1 Context & Technology Choice

Five classification algorithms were selected and trained. K-Nearest Neighbors(KNN), XGBoost, Random Forest, Decision Tree, and Logistic Regression. Each model was separately trained on the entire feature set as well as three feature subsets. These subsets were derived from Lasso regularization, Random Forest feature selection, and XGBoost feature selection.

11.2 Justification

The classification models selected were chosen from class material to exemplify strong understanding of the course. Training each model on all subsets of features allows for direct comparison of regularization techniques rather than choosing a universal best. A few variations stood out such as XGBoost with RF selection, Random Forest with XGB selection, and random forest with lasso regularization. Ultimately we chose XGBoost with Random Forest feature selection due to its strong recall score. We prioritized recall to ensure the model flags as many true threats as possible. The tradeoff for an increased number of false positives is warranted given the use case.

11.3 Status

Jackson Horan: Proposed, 4/24/2026

Jackson Horan: Accepted, 4/24/2026

Completed on 4/27/2026

12. Model Evaluation

12.1 Context & Technology Choice

The optimized model was evaluated with a test dataset and validation dataset of equal size. We used metrics such as accuracy, F1, recall, and precision. Additionally we used a confusion matrix and an ROC curve to assess AUC. The XGBoost model was evaluated across two datasets, a test dataset and a validation dataset to ensure that the results reflect the models

true predictive capabilities. The accuracy score provides an overall view of the model correctness. Precision score provides a proportion of the positive predictions that were truly positive. Recall gives a proportion of the actual positives correctly identified. F1 score is a balance of both recall and precision.

12.2 Context & Technology Choice

As shown in figure 17 the model scored well on both the test and validation datasets respectively.

```
XGBoost with RF Feature Selection (Test)
-----
Accuracy: 0.9999
Precision: 0.9998
Recall: 1.0000
F1-score: 0.9999

XGBoost with RF Feature Selection (Validation)
-----
Accuracy: 0.9993
Precision: 0.9994
Recall: 0.9998
F1-score: 0.9996
```

Figure 17: Evaluation Metrics

While the model performs slightly worse on the validation dataset, this difference is negligible and likely due to random sampling variations. The confusion matrix and ROC curve for the test and validation data sets are shown in figures 18 through 21. The small differences in performance can be seen in the confusion matrix as just a few single data points. The AUC score for both the test and validation datasets is 1.

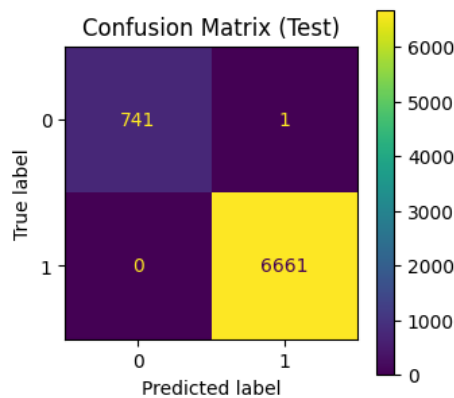


Figure 18: Confusion Matrix (Test)

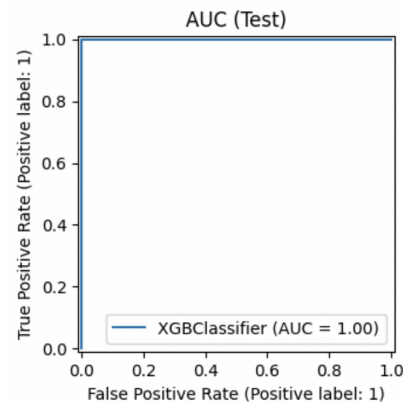


Figure 19: ROC Curve and AUC (Test)

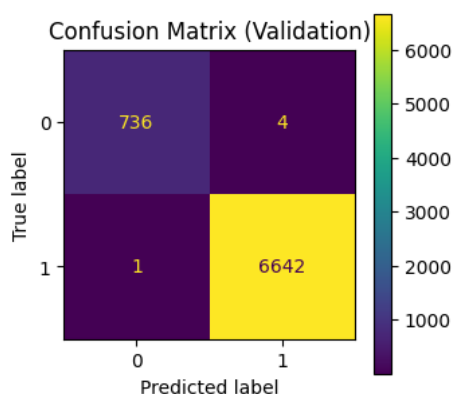


Figure 20: Confusion Matrix (Validation)

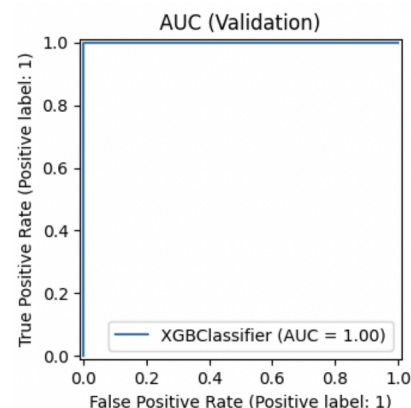


Figure 21: ROC Curve and AUC (Validation)

12.3 Status

Jackson Horan: Proposed, 4/27/2026

Jackson Horan: Accepted, 4/27/2026

Completed on 4/30/2026

14. Concluding Remarks and Project Summary

14.1: Context & Technology Choice

This project focused on accurately detecting denial-of-service (DoS) attacks while building a machine learning pipeline that is scalable and easy to reproduce. A structured ETL pipeline was used to consistently load data from multiple formats (CSV, JSON, and Parquet), which improved reliability and organization among our group. Data cleaning, standardization, and handling of missing or unimportant values helped reduce noise in the dataset and improved overall model performance.

An important architectural decision was the use of multiple feature selection techniques, including L1-regularization (Lasso) and tree-based importance methods (Random Forest & XGBoost). This approach reduced dimensionality while maintaining statistically significant predictors. This helped balance model complexity and interpretability. Additionally, splitting the data into training, validation, and testing sets with stratification addressed class imbalance and prevented data leakage.

Given the project's use case of prioritizing the detection of potential attacks, the selection of the XGBoost model with Random Forest feature selection proved to be the most effective. This combination achieved strong recall while maintaining high overall performance. This ensured that true threats were identified even at the cost of increased false positives. This trade-off seemed to be appropriate for a cybersecurity context, since missing an attack could be more costly than raising extra alerts.

Overall, the architecture we implemented performed well in terms of predictive accuracy, robustness, and generalizability. The pipeline design allows for easy updates, such as incorporating new data sources (Other file formats) or testing additional models, which supports long-term maintainability. However, there are areas for improvement. Future iterations could explore more advanced techniques for handling class imbalance as well as more hyperparameter tuning and model ensembling strategies. To conclude, the final system we have developed demonstrates a strong balance between performance and practicality. We've learned important lessons about trade-offs, feature selection, and the importance of aligning model evaluation metrics with real-world objectives.

14.2 Justifications

The project was designed to detect DoS attacks with high recall while still being scalable and reproducible. A structured ETL pipeline was created to create consistent ingestion of multi-format data and to improve reliability and communication among our group. Data preprocessing was done to reduce noise and enhance model performance.

Feature selection methods such as Lasso (for sparsity and interpretability), Random Forest and XGBoost (for non-linear relationships) were chosen to use, resulting in a robust feature set. Stratified data splitting was done to prevent leakage and address class imbalance.

XGBoost with Random Forest feature selection was selected due to its strong recall, aligning with our goal of minimizing missed attacks. The ADR we have developed supports maintainability and future improvements, such as advanced imbalance handling and further model tuning.

14.3 Status

Sania Ali: Proposed, 4/28/26

Sania Ali, Madeline Geittmann, Jackson Horan, Lara Seledotis: Accepted, 4/28/26

Completed & passed onto team, 4/28/26

15. References

- [1] America's Cyber Defense Agency, "Understanding Denial of Service Attacks," Cybersecurity & Infrastructure Security Agency, February 01 2021, [Online]. Available: <https://www.cisa.gov/news-events/news/understanding-denial-service-attacks>
- [2] Canadian Institute for Cybersecurity, "Intrusion Detection Evaluation Dataset," University of New Brunswick, CIC-IDS2017, July 2017, [Online]. Available: <https://www.unb.ca/cic/datasets/ids-2017.html>
- [3] "Data Transformations in Machine Learning," Geeks for Geeks, July 23, 2025, [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/data-transformation-in-machine-learning/>
- [4] "EDA – Exploratory Data Analysis in Python," Geeks for Geeks, April 6, 2026, [Online]. Available: <https://www.geeksforgeeks.org/data-analysis/exploratory-data-analysis-in-python/>
- [5] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization", 4th International Conference on Information Systems Security and Privacy (ICISSP), Portugal, January 2018.
- [6] "K means Clustering – Introduction," Geeks for Geeks, November 10, 2025, [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/k-means-clustering-introduction/>
- [7] "Matplotlib Subplot," W3 Schools, [Online]. Available: https://www.w3schools.com/python/matplotlib_subplot.asp
- [8] M. Mohaisen, "In-class Lab 2: Extract, Transform, and Load (ETL)," COMP379, Loyola University Chicago, Chicago, IL, 2026, [Google Colab Notebook]. Available: https://drive.google.com/file/d/11I432g8uAKu5BEW2PunpGt7qXpp0VAP_/view
- [9] M. Mohaisen, "In-class Lab 8: Polynomial Regression," COMP379, Loyola University Chicago, Chicago, IL, 2026, [Google Colab Notebook]. Available: <https://drive.google.com/file/d/1115SXLrf8e3BIkkPV0jWGAEy73ytU0o6/view>
- [10] M. Mohaisen, "Module 5: Supervised Learning Classification," COMP379, Loyola University Chicago, Chicago, IL, 2026, [Google Colab Notebook]. Available: <https://drive.google.com/file/d/1NKyUn9an4eSsglpEfkAzoVha97kj0ChX/view>

- [11] M. Mohaisen, “In-class Lab 10: Classification using the Perceptron, Logistic Regression, and Decision Trees Models,” COMP379, Loyola University Chicago, Chicago, IL, 2026, [Google Colab Notebook]. Available:
https://drive.google.com/file/d/1INSNwnVFViWH_5kLgFT_SHMANDL4urp7/view
- [12] M. Mohaisen, “In-class Lab 11: Boosting Classifiers: XGBoost, LightGBM, and CatBoost,” COMP379, Loyola University Chicago, Chicago, IL, 2026, [Google Colab Notebook]. Available:
https://drive.google.com/file/d/1-uu-ecjFGPaeL_XYqdUPcOnNRtxbh9Q4/view
- [13] M. Mohaisen, “In-class Lab 12: Hyperparameter Tuning,” COMP379, Loyola University Chicago, Chicago, IL, 2026, [Google Colab Notebook]. Available:
https://drive.google.com/file/d/1bgHtlCkAcwmU1VB4gxgBrCJWs57_Pjv/view
- [14] “RocCurveDisplay,” Scikit Learn, 2026, [Online]. Available:
<https://scikit-learn.org/stable/modules/generated/sklearn.metrics.RocCurveDisplay.html>
- [15] “What is an intrusion detection system (IDS)?” IBM, [Online]. Available:
<https://www.ibm.com/think/topics/intrusion-detection-system>